

Análisis del comportamiento del sistema operativo Linux mediante un demonio de monitoreo con systemd

Análisis reflexivo y justificación teórica de las observaciones

Autora:	Cindy Guzmán
Equipo:	Dell Inspiron 15 3520 (host cindy-guzman-Inspiron-15-3520)
Fecha de la sesión:	29 de septiembre de 2026
Ventana de observación:	22:52:50 a 23:01:24 (CST)
Servicio analizado:	mi-demonio.service
Registros analizados:	78 líneas de bitácora

1. Resumen

Este reporte documenta y analiza el comportamiento de un sistema Linux durante una sesión de trabajo en la que se editó, se dotó de permisos de ejecución y se puso en marcha un demonio de monitoreo administrado por **systemd**. El demonio registra cada cinco segundos el número de procesos del sistema y la memoria RAM disponible en el archivo `bitacora_demonio.txt`. Se analizaron 78 registros correspondientes a un intervalo de aproximadamente ocho minutos y medio, junto con los resultados de los comandos ejecutados.

Los hallazgos principales son: (a) el sistema se mantuvo estable, con entre 318 y 338 procesos y un servicio que consume aproximadamente 0.7 % de CPU y 2.9 MB de memoria; (b) la RAM disponible presenta un patrón de *mesetas* con escalones de 300 a 360 MB asociados a la apertura y el cierre de aplicaciones, no a fugas de memoria; (c) la bitácara muestra pequeñas desviaciones respecto del intervalo nominal de 5 s (deriva temporal) y una línea duplicada causada por la ejecución simultánea del script manual y del servicio; y (d) el uso de `sudo gedit` y de `chmod 777` dejó archivos con propietario *root* y permisos más amplios de lo necesario. Cada observación se justifica con los conceptos teóricos de procesos, gestión de memoria, planificación, control de acceso y concurrencia.

2. Objetivos

Objetivo general. Interpretar el comportamiento observable del sistema operativo durante la ejecución de un demonio de monitoreo, justificando cada observación con fundamentos teóricos.

Objetivos específicos.

- Describir la secuencia de comandos ejecutados y sus resultados.
- Caracterizar estadísticamente las métricas registradas (procesos y RAM disponible).
- Explicar los eventos anómalos o notables (caídas de memoria, picos de procesos, duplicados, deriva temporal).
- Identificar riesgos de seguridad y de buenas prácticas, y proponer mejoras.

3. Entorno y metodología

La información proviene exclusivamente de la salida de la terminal capturada durante la sesión. La Tabla 1 resume los elementos involucrados.

Elemento	Descripción
Sistema	Linux con systemd como sistema de inicio (PID 1); interfaz en español; editor gedit.
Script	<code>~/mi_demonio.sh</code> (337 bytes): bucle infinito que escribe una línea por iteración.
Servicio	<code>/etc/systemd/system/mi-demonio.service</code> , «Mi primer demonio», activo desde las 22:41:21.
Bitácara	<code>~/bitacora_demonio.txt</code> ; formato: fecha Procesos: N RAM disponible: M MB.
Intervalo nominal	5 segundos.
Herramientas de observación	<code>ls -l</code> , <code>cat</code> , <code>systemctl status</code> , <code>tail -f</code> .

Tabla 1. Elementos del entorno de observación.

Limitaciones. El contenido exacto de `mi_demonio.sh` y de la unidad de `systemd` no formó parte de la salida capturada, por lo que el análisis del mecanismo interno se apoya en el formato de la bitácara y en lo que

`systemctl status` reporta. Los registros son dos segmentos separados (44 líneas de `cat` y 34 de `tail -f`) con un intervalo sin captura de 136 s. La marca de tiempo tiene resolución de un segundo. La bitácora comienza a las 22:52:50, es decir, 11 minutos después de iniciado el servicio, lo que indica que el archivo fue reiniciado o eliminado antes; los datos no permiten determinar la causa. Las atribuciones de causa que se proponen en la sección 5 son hipótesis consistentes con los datos, no comprobaciones.

4. Resultados

4.1 Secuencia de comandos y resultados

N.º	Comando	Resultado observado
1	<code>sudo gedit ~/mi_demonio.sh</code>	Pide contraseña y abre gedit. Aparecen avisos (Peas, PeasGtk, gtksourceview, dconf/dbus-launch) sin efecto sobre el script.
2	<code>ls -l</code>	<code>mi_demonio.sh</code> : -rw-r--r-- root root, 337 bytes (sin permiso de ejecución). <code>bitacora_demonio.txt</code> : 1638 bytes.
3	<code>sudo chmod 777 ~/mi_demonio.sh</code>	Sin salida (éxito).
4	<code>ls -l</code>	<code>mi_demonio.sh</code> : -rwxrwxrwx root root. La bitácora creció a 2016 bytes (22:55).
5	<code>~/mi_demonio.sh</code>	Ejecución manual interrumpida con Ctrl+C.
6	<code>cat ~/bitacora_demonio.txt</code>	44 líneas (22:52:50 a 22:56:21); una línea duplicada (22:56:05 y 22:56:06).
7	<code>sudo gedit /etc/systemd/system/mi-demonio.service</code>	Abre la unidad de systemd; mismos avisos inofensivos.
8	<code>sudo systemctl daemon-reload</code>	Sin salida (éxito).
9	<code>systemctl status mi-demonio</code>	enabled; active (running) desde 22:41:21; PID 17028 (bash) y 22080 (sleep 5); Tasks: 2; Memory: 2.9M (pico 5.9M); CPU: 7.307 s.
10	<code>tail -f ~/bitacora_demonio.txt</code>	34 líneas visibles (22:58:37 a 23:01:24) mientras el servicio sigue escribiendo.

Tabla 2. Comandos ejecutados con sus resultados.

4.2 Estadísticas descriptivas de la bitácora

Métrica	Mínimo	Máximo	Media	Mediana	Desv. est.
Procesos (n = 78)	318	338	325.2	323	5.1
RAM disponible, MB (n = 78)	11 188	11 596	11 365	11 260	156.6

Tabla 3. Estadísticos de las métricas registradas. Rango de la RAM: 408 MB. Correlación de Pearson entre procesos y RAM: $r = -0.16$.

La mediana de la RAM (11 260 MB) queda muy por debajo de su máximo, y la desviación estándar es alta en relación con la variación de los procesos. Esto refleja que los datos se agrupan en dos niveles (aprox. 11.2 GB y 11.5 a 11.6 GB) y no en una distribución continua alrededor de la media (Figura 2).

Figura 1. Evolución de la RAM disponible y del número de procesos

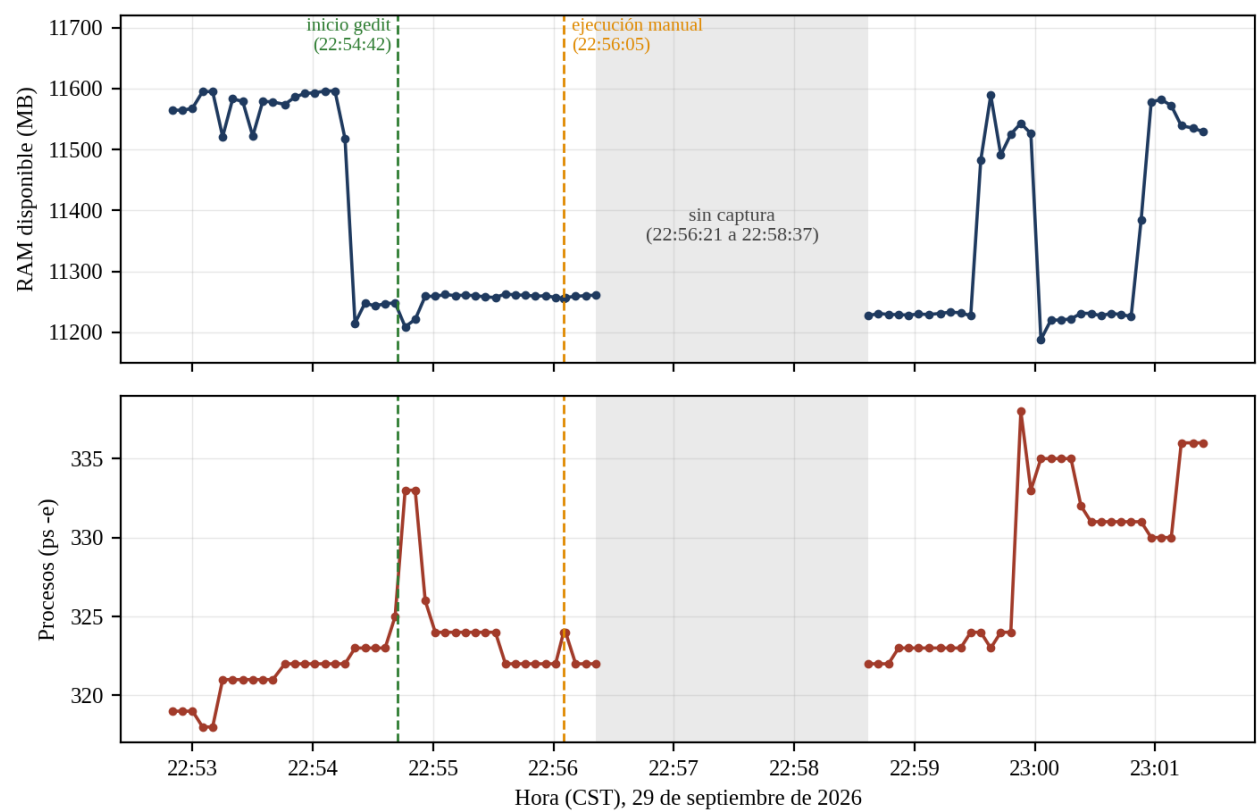


Figura 1. Series de tiempo de la RAM disponible y del número de procesos. Las líneas discontinuas marcan el inicio de gedit (22:54:42) y la ejecución manual (22:56:05); la franja gris es el intervalo sin captura.

Figura 2. Relación y distribución de las métricas

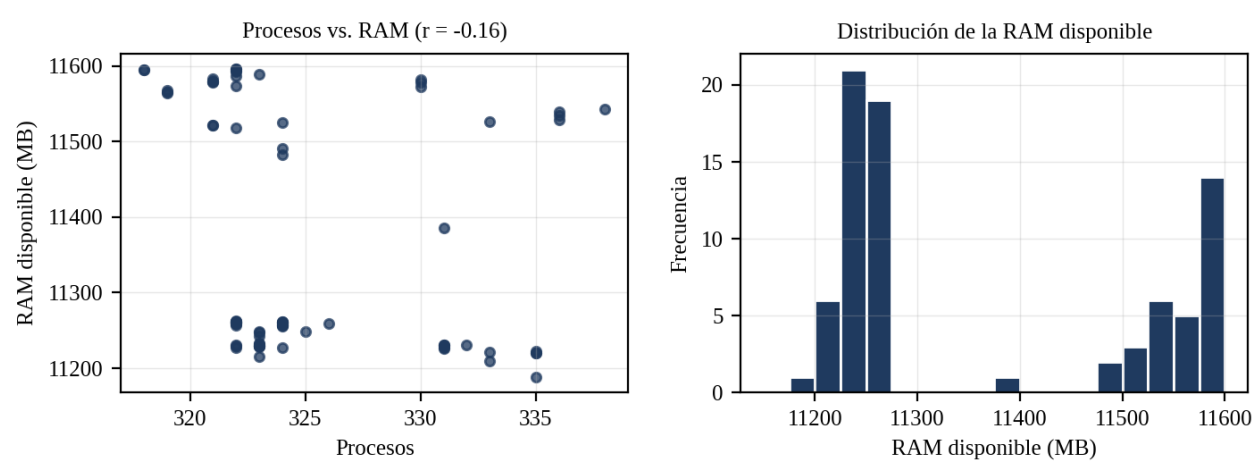


Figura 2. Dispersión procesos-RAM (correlación débil) e histograma de la RAM disponible (dos concentraciones).

4.3 Eventos notables

Hora	Observación
22:54:16 a 22:54:21	La RAM disponible cae 303 MB (11 518 a 11 215) con un solo proceso más. Ocurre unos 20 s antes del primer aviso de gedit (22:54:42).

Hora	Observación
22:54:41 a 22:54:51	Los procesos suben de 323 a 333 (+10) durante el arranque de gedit con sudo; vuelven a 326 (22:54:56) y 324 (22:55:01). Coincide con los avisos de dconf de las 22:54:54.
22:56:05 y 22:56:06	Dos líneas consecutivas: la del script manual y la del servicio. Ambas muestran 324 procesos, dos más que la línea anterior (322).
22:56:21 a 22:58:37	Sin captura (136 s). Incluye la apertura del segundo gedit (22:57:17) y daemon-reload.
22:59:28 a 22:59:38	La RAM sube 362 MB (11 227 a 11 589), regresando al nivel inicial de la sesión.
22:59:53 a 23:00:03	Máximo de procesos (338) y luego mínimo de RAM (11 188 MB, a las 23:00:03); la RAM cae 355 MB respecto de 11 543 MB.
23:00:53 a 23:00:58	La RAM se recupera de 11 385 a 11 578 MB (+193 MB en 5 s); los procesos bajan a 330.
23:01:13	Procesos 330 a 336 (+6) con una caída de RAM de solo 33 MB.

Tabla 4. Eventos identificados en la bitácora.

5. Análisis reflexivo y justificación teórica

5.1 El demonio como proceso administrado por systemd

Un *demonio* es un proceso de larga duración que se ejecuta en segundo plano, sin terminal de control, y presta un servicio al sistema (Silberschatz et al.; Kerrisk). En Linux moderno, **systemd** es el proceso con PID 1: arranca los servicios, los supervisa y los agrupa en *cgroups* (grupos de control) que permiten contabilizar y limitar recursos. La salida de `systemctl status` lo evidencia: el servicio aparece en su propio CGroup (`/system.slice/mi-demonio.service`) con contabilidad propia de tareas (2, con un límite de 18 132), memoria (2.9 MB, pico de 5.9 MB) y CPU (7.307 s). El estado *enabled* significa que systemd creó un enlace en el destino de arranque correspondiente, de modo que el servicio se inicia con el sistema; *active (running)* indica que el proceso principal sigue vivo.

Los dos procesos del cgroup (`bash` y `sleep 5`) ilustran el modelo de creación de procesos de UNIX: el intérprete llama a `fork()` para clonarse y `exec()` para reemplazar la copia por el programa deseado. Cada comando externo del bucle (por ejemplo `sleep`) es un proceso hijo efímero, y el estado bloqueado del padre mientras espera a su hijo explica que el servicio casi no consuma CPU.

5.2 Consumo de CPU y memoria del propio servicio

Con 7.307 s de CPU acumulados en aproximadamente 17 minutos (unos 1 020 s), la utilización media es cercana a 0.7 %. Suponiendo una iteración cada cinco segundos (alrededor de 204 iteraciones), el costo sería de unos 36 ms de CPU por iteración. Es coherente con un bucle que pasa casi todo su tiempo dormido: un proceso bloqueado en `sleep` no compite por la CPU, y el planificador solo lo despierta cuando vence el temporizador (Tanenbaum y Bos; Love). El costo por iteración se explica por la creación de los procesos auxiliares que consultan el estado del sistema, cada uno con su `fork()`, `exec()`, lectura del sistema de archivos `/proc` y terminación. Leer directamente `/proc` desde el propio script reduciría este costo.

5.3 Deriva temporal del intervalo de muestreo

Aunque el intervalo nominal es de 5 s, la bitácora muestra saltos de 6 s (22:53:40 a 22:53:46, 22:59:07 a 22:59:13 y 23:01:13 a 23:01:19). En cada segmento el promedio real fue de 5.02 s (211 s en 42 intervalos) y 5.06 s (167 s en 33 intervalos), con resolución de un segundo. La causa teórica es que el período efectivo es *tiempo de dormir + tiempo de ejecución de la iteración*: `sleep 5` garantiza una espera *mínima* y no un ritmo exacto, además de la latencia de planificación con la que el sistema devuelve la CPU al proceso. Estas decenas

de milisegundos se acumulan hasta que, al truncar a segundos, aparece un salto de 6 s. Si se requiriera periodicidad estricta, convendría un temporizador de systemd (*timer*) o calcular el tiempo restante contra un reloj monótono.

5.4 Memoria: qué mide realmente «RAM disponible»

Linux utiliza la memoria libre como caché de páginas (archivos leídos recientemente) y como buffers para acelerar la E/S. Por ello la cifra de memoria «libre» suele ser engañosamente baja. La métrica **MemAvailable** (columna *available* de *free*, disponible desde el kernel 3.14) estima cuánta memoria puede entregarse a nuevas aplicaciones sin recurrir a intercambio (*swap*), sumando la memoria libre y la parte recuperable de cachés (*proc(5)*). Es, por tanto, el indicador correcto para monitoreo.

Los datos muestran un comportamiento en **escalones**: mesetas estables (variaciones de pocos MB, por ejemplo 11 256 a 11 263 MB entre 22:54:56 y 22:56:21) separadas por saltos de 300 a 360 MB. Esta forma es característica de aplicaciones que reservan memoria anónima en ráfagas al iniciar y la devuelven al terminar, mientras que un patrón de fuga de memoria sería una pendiente descendente sostenida, que no se observa. Dentro de una meseta, las oscilaciones de unos MB se explican por actividad del kernel, la caché y los procesos efímeros del propio demonio.

La caída de 303 MB de las 22:54:21 ocurre antes del arranque de gedit (22:54:42) y no se revirtió cuando gedit se cerró (la RAM quedó en aprox. 11 260 MB), lo que impide atribuirla a gedit. Lo más consistente con los datos es que otra aplicación del escritorio, que permaneció abierta, reservó esa memoria. Recuperó su nivel inicial (aprox. 11 589 MB) a las 22:59:38, y un segundo episodio semejante (caída de 355 MB a las 23:00:03 y recuperación a las 23:00:58) acompañado de un pico de procesos sugiere un mismo tipo de aplicación multiproceso que se abre y se cierra. Los datos no permiten identificarla; esto es correlación temporal, no prueba de causalidad. La correlación global entre procesos y RAM es débil ($r = -0.16$): más procesos no implica necesariamente menos memoria, pues pesan mucho más las aplicaciones grandes que la cantidad de procesos pequeños.

5.5 El conteo de procesos

El número de procesos refleja la actividad de creación y terminación de tareas. Incluye procesos de usuario y de sistema (los hilos del kernel aparecen como procesos), pero no los hilos de cada proceso. La línea base de 318 a 325 corresponde al sistema en reposo con sesión gráfica. El pico de +10 durante el arranque de gedit con *sudo* se explica porque *sudo* crea su propio proceso, gedit inicia procesos auxiliares y, al ejecutarse como *root*, se generan además intentos de conexión al bus de sesión (origen de los avisos de *dbus-launch*). Que el conteo vuelva a su nivel previo tras cerrar la aplicación confirma que no quedaron procesos huérfanos.

5.6 Concurrencia: la línea duplicada

A las 22:56:05 y 22:56:06 coexistieron dos instancias del mismo script (la manual y el servicio) que añadían texto al mismo archivo. Al abrir un archivo en modo *append* (>>, bandera *O_APPEND*), el kernel posiciona cada escritura al final de forma atómica, por lo que las líneas no se sobrescriben ni se corrompen; sin embargo, se *intercalan* y se duplican los registros. Además, ambas instancias se contaron mutuamente en la métrica de procesos (324 frente a 322), de modo que la medición perturbó lo medido, un efecto común en monitoreo. La solución teórica es *exclusión mutua*: garantizar una sola instancia mediante un cerrojo de archivo (*flock*) o dejar que solo el servicio ejecute el script.

5.7 Permisos y control de acceso

Linux aplica control de acceso discrecional (DAC): cada archivo tiene propietario, grupo y tres tríos de permisos (lectura, escritura, ejecución). Al guardar con `sudo gedit`, el archivo se creó con propietario `root` y modo `-rw-r--r--` (sin bit de ejecución), lo que impediría ejecutarlo directamente hasta aplicar `chmod`. Los archivos `mundo.sh` y `salud.sh` presentan el mismo patrón (`root` y modo `777`).

El modo `777` concede escritura a cualquier usuario del sistema. Es innecesario (`755` basta para que el dueño edite y todos ejecuten) y viola el *principio de mínimo privilegio*: cualquier cuenta local podría modificar un script que otro proceso ejecuta. Si el servicio corriera con privilegios elevados (no verificable con los datos disponibles), esto sería una vía de escalada de privilegios. Los avisos de `gedit` (bibliotecas `Peas` ausentes, esquemas de estilo `Yaru` incompatibles con la versión de `gtksourceview` y ausencia de `dbus-launch` para `dconf`) no afectan el resultado, pero son síntoma de ejecutar una aplicación gráfica como `root`, práctica que se desaconseja porque esa aplicación tiene acceso irrestricto al sistema y puede dejar archivos de configuración con propietario incorrecto.

5.8 Persistencia de la bitácora y otras consideraciones

Cada línea ocupa 63 bytes (1 638 bytes = 26 líneas y 2 016 bytes = 32 líneas, lo cual concuerda con los tamaños vistos en `ls -l`). A 12 líneas por minuto, el crecimiento es de unos 1.1 MB al día (aprox. 33 MB al mes): pequeño, pero ilimitado si no se rota el archivo. Además, `bash` lee los scripts de manera incremental, por lo que editar un script mientras se ejecuta puede provocar comportamientos impredecibles; por eso, tras modificarlo debe reiniciarse el servicio (`systemctl restart`), y `daemon-reload` solo se requiere cuando cambia la *unidad*.

6. Conclusiones

- El demonio funciona correctamente y de forma ligera: aproximadamente 0.7 % de CPU y menos de 6 MB de memoria, en línea con un proceso que permanece casi siempre bloqueado.
- La RAM disponible varía por escalones vinculados a la actividad de aplicaciones, no por fugas; la métrica `MemAvailable` es la adecuada porque incorpora la memoria recuperable de la caché.
- Los picos del número de procesos son transitorios y se revierten, lo que indica una gestión correcta de la terminación de procesos.
- El intervalo de 5 s presenta una deriva de decenas de milisegundos por iteración, inherente al esquema «ejecutar y dormir».
- La duplicación de registros es un ejemplo práctico de condición de carrera evitable con exclusión mutua.
- El uso de `sudo gedit` y de `chmod 777` funciona, pero genera archivos con propietario `root` y permisos excesivos: una deuda de seguridad y mantenimiento.

7. Recomendaciones

- Editar sin privilegios (`gedit ~/mi_demonio.sh`) o recuperar la propiedad con `sudo chown cindy-guzman:cindy-guzman ~/mi_demonio.sh`; usar `chmod 755`.
- Ejecutar el script únicamente a través del servicio y reiniciarlo con `sudo systemctl restart mi-demonio` tras cada cambio.
- Añadir un cerrojo (`flock`) para impedir instancias simultáneas.
- Leer `/proc/meminfo` (`MemAvailable`) directamente para reducir procesos auxiliares y mejorar la exactitud.

- Registrar también la marca de tiempo con mayor resolución y considerar un temporizador de systemd si se necesita periodicidad estricta.
- Configurar rotación de la bitácora (`logrotate`) o enviar la salida al *journal* para evitar crecimiento ilimitado.

Referencias

Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*. Wiley.

Tanenbaum, A. S. y Bos, H. *Modern Operating Systems*. Pearson.

Love, R. *Linux Kernel Development*. Addison-Wesley.

Kerrisk, M. *The Linux Programming Interface*. No Starch Press.

Páginas de manual de Linux: `systemd.service(5)`, `systemd.resource-control(5)`, `cgroups(7)`, `proc(5)`, `free(1)`, `ps(1)`, `chmod(1)`, `sudo(8)`, `bash(1)`, `open(2)`, `fork(2)`, `execve(2)` y `flock(1)`.

Documentación del kernel de Linux: *Control Group v2*.

Anexo A. Registros completos de la bitácora

Los 78 registros analizados (segmento 1: cat; segmento 2: tail -f). La marca * corresponde a la ejecución manual.

N.º	Hora	Proc.	RAM (MB)	N.º	Hora	Proc.	RAM (MB)
1	22:52:50	319	11565	40	22:56:05*	324	11256
2	22:52:55	319	11564	41	22:56:06	324	11257
3	22:53:00	319	11568	42	22:56:11	322	11259
4	22:53:05	318	11595	43	22:56:16	322	11260
5	22:53:10	318	11595	44	22:56:21	322	11261
6	22:53:15	321	11521	45	22:58:37	322	11227
7	22:53:20	321	11583	46	22:58:42	322	11231
8	22:53:25	321	11579	47	22:58:47	322	11229
9	22:53:30	321	11522	48	22:58:52	323	11229
10	22:53:35	321	11579	49	22:58:57	323	11228
11	22:53:40	321	11578	50	22:59:02	323	11230
12	22:53:46	322	11574	51	22:59:07	323	11229
13	22:53:51	322	11586	52	22:59:13	323	11231
14	22:53:56	322	11592	53	22:59:18	323	11233
15	22:54:01	322	11593	54	22:59:23	323	11232
16	22:54:06	322	11596	55	22:59:28	324	11227
17	22:54:11	322	11596	56	22:59:33	324	11482
18	22:54:16	322	11518	57	22:59:38	323	11589
19	22:54:21	323	11215	58	22:59:43	324	11491
20	22:54:26	323	11248	59	22:59:48	324	11525
21	22:54:31	323	11243	60	22:59:53	338	11543
22	22:54:36	323	11247	61	22:59:58	333	11526
23	22:54:41	325	11248	62	23:00:03	335	11188
24	22:54:46	333	11209	63	23:00:08	335	11220
25	22:54:51	333	11221	64	23:00:13	335	11220
26	22:54:56	326	11259	65	23:00:18	335	11222
27	22:55:01	324	11259	66	23:00:23	332	11231
28	22:55:06	324	11262	67	23:00:28	331	11231
29	22:55:11	324	11260	68	23:00:33	331	11227
30	22:55:16	324	11261	69	23:00:38	331	11231
31	22:55:21	324	11260	70	23:00:43	331	11229
32	22:55:26	324	11258	71	23:00:48	331	11226
33	22:55:31	324	11257	72	23:00:53	331	11385
34	22:55:36	322	11263	73	23:00:58	330	11578
35	22:55:41	322	11261	74	23:01:03	330	11582
36	22:55:46	322	11261	75	23:01:08	330	11572
37	22:55:51	322	11259	76	23:01:13	336	11539
38	22:55:56	322	11260	77	23:01:19	336	11535
39	22:56:01	322	11257	78	23:01:24	336	11529